PAI-2 BYODSEC

Bring Your Own Device seguro mediante VPN SSL «road warrior» (TLS 1.3 + mTLS) para la Universidad U-Secure

Grupo	ST-2
Integrantes	Raimundo García Pérez · Konstantinos Peridis · Luis De Alarcón Conradi
Asignatura	Seguridad en Sistemas de Información e Internet (SSII) · ETSII · Universidad de Sevilla
Repositorio	github.com/luialacon/ssii_PAI2 (rama raimundo_dev)
Fecha	21/10/2026

1. Resumen

Hemos construido un **gateway VPN y un cliente BYOD sobre sockets TLS 1.3 nativos** (módulo `ssl` de Python sobre OpenSSL) que cumplen la política de la universidad: *solo usuarios, dispositivos y servicios previamente autorizados acceden a los recursos corporativos, por canales que garantizan confidencialidad, integridad y autenticidad*. Elegimos sockets frente a REST/gRPC porque dan control total del handshake y dejan el .pcap sin ruido de un servidor web.

Decisiones técnicas y algorítmicas

Requisito	Decisión	Por qué
RS1 PKI + mTLS	CA interna ECDSA P-384 (*pathlen=0*); certificados por CSR; CRL; `CERT_REQUIRED`	El servidor exige el carnet del dispositivo; el cliente valida CA **y nombre** del servidor
RS2 TLS 1.3	*min* = *max* = *TLS 1.3*; suites **TLS_AES_256_GCM_SHA384** y **TLS_CHACHA20_POLY1305_SHA256**; sin tickets	Todas las suites TLS 1.3 son AEAD con PFS (ECDHE obligatorio); sin tickets no sobrevive material de sesión al cierre
RF2 doble capa	mTLS + usuario/contraseña (**Argon2id**) + **CN del certificado = usuario**	Un carnet robado no sirve sin contraseña, ni para entrar como otro usuario
RF1 inmutabilidad	Triggers SQLite + log de auditoría con **cadena de hashes**	La identidad y la clave pública no se pueden alterar; manipular el log se detecta
RS3 300 usuarios	Hilo por conexión, semáforo de Argon2id, benchmark de 300 sesiones simultáneas **plano frente a TLS**	Misma lógica de servidor en ambos: la diferencia medida es el cifrado
RS4 Auditoría	4 capturas `.pcapng` analizadas con tshark + 52 eventos auditados	Demuestra tráfico 100 % cifrado y rastrea conexiones, rechazos y desconexiones

Estado: 37 pruebas automáticas en verde, 11 ataques simulados bloqueados de 11 y benchmark de 300 sesiones simultáneas con 300/300 éxitos. Código: 1448 líneas en `src/`, 2153 en pruebas, benchmark y ataques.

2. Arquitectura

El proyecto sigue el escenario de la Figura 2 del enunciado: dispositivos personales que cruzan Internet hasta un gateway que da acceso a la red corporativa. Todo el tráfico va dentro de un único túnel TLS 1.3; el atacante solo ve bytes cifrados.

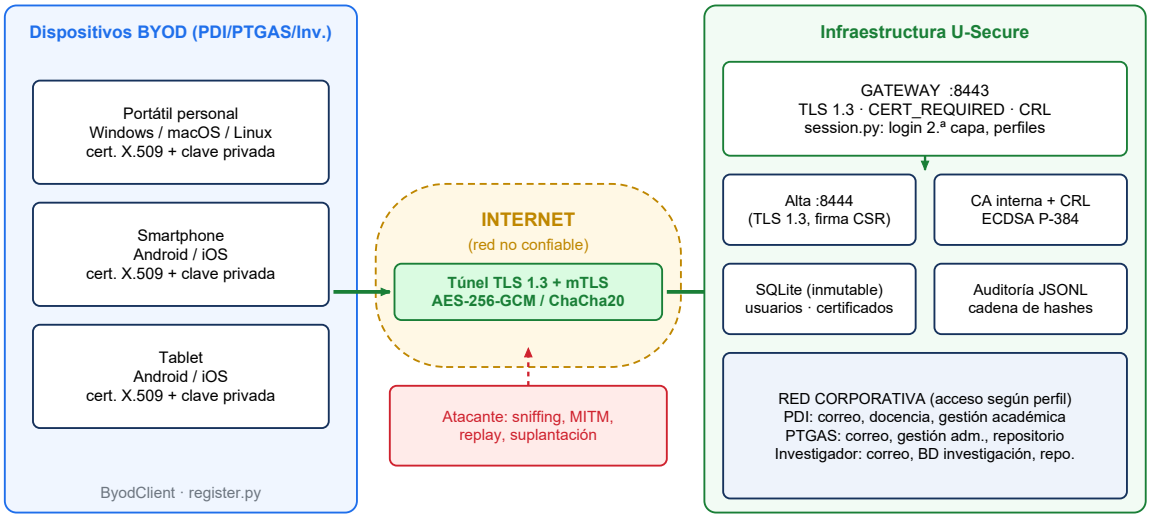


Figura 1. Arquitectura cliente-servidor/BYOD y componentes. El servicio de alta (:8444) usa TLS 1.3 sin certificado de cliente porque el usuario nuevo aún no tiene carnet.

2.1 Negociación mTLS y doble capa de autenticación

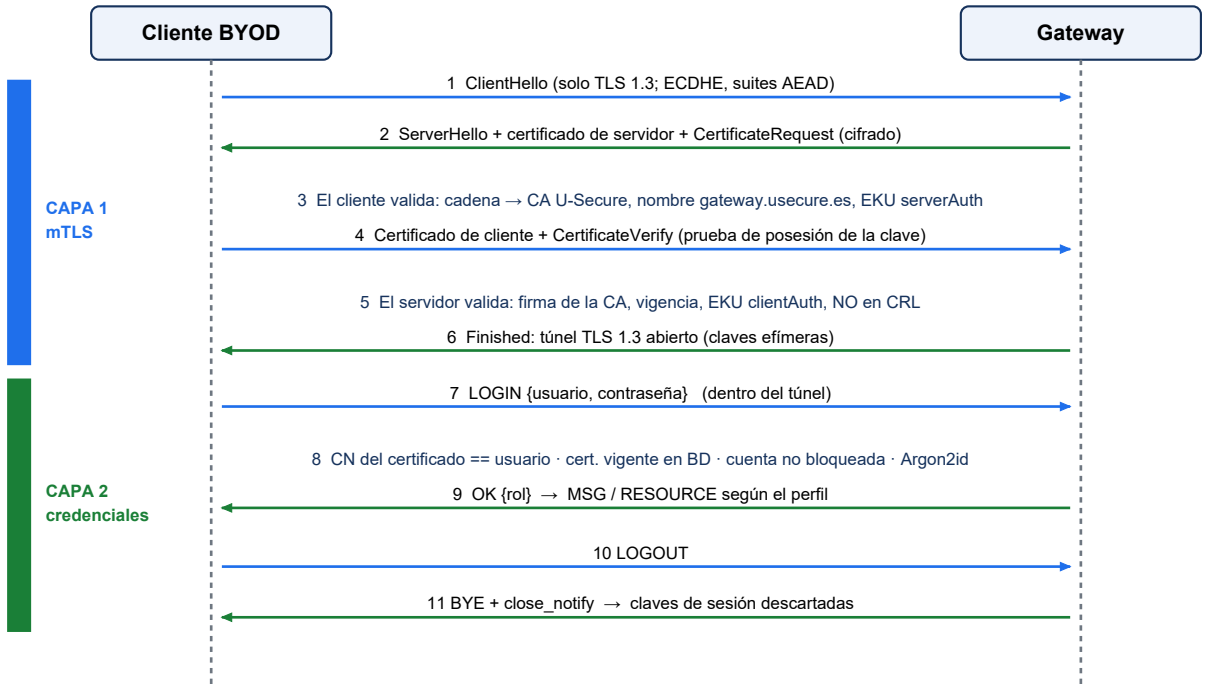


Figura 2. Handshake TLS 1.3 con autenticación mutua (pasos 1-6) y autenticación de aplicación (7-11). En TLS 1.3 el certificado del servidor ya viaja cifrado.

2.2 Ciclo de vida de los certificados X.509

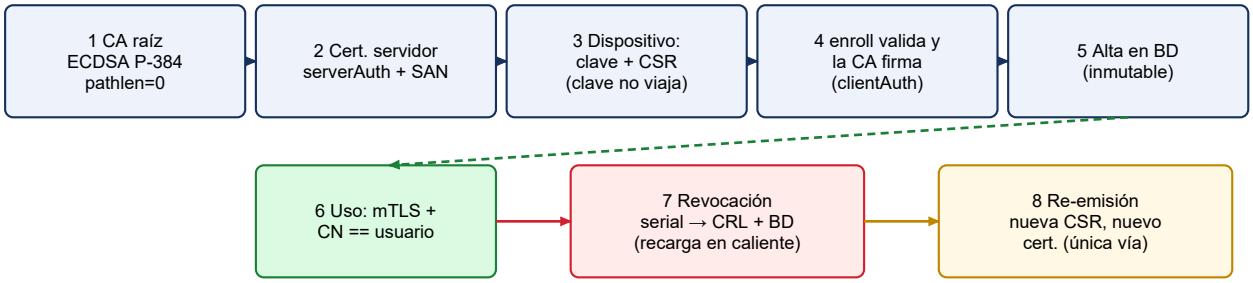


Figura 3. De la CA al uso, revocación y re-emisión. La clave privada del usuario se genera en su dispositivo y **nunca** viaja: solo la CSR.

3. Justificación criptográfica

3.1 PKI interna (RS1)

La **CA raíz** es autofirmada, ECDSA con curva P-384 y SHA-384, con *BasicConstraints CA:TRUE pathlen=0* y *KeyUsage keyCertSign + cRLSign*: solo puede firmar certificados finales, no otras CA. El **certificado del servidor** (P-384) lleva EKU *serverAuth* y SAN (*gateway.usecure.es*, *localhost*, 127.0.0.1). Los **certificados de usuario** (P-256) llevan EKU *clientAuth*, *KeyUsage digitalSignature*, identificadores SKI/AKI y punto de distribución de CRL; el CN es el nombre de usuario y la OU, el rol.

Emisión por CSR. El dispositivo genera su par de claves y envía solo la CSR. La CA comprueba la firma de la CSR y que el CN coincida con el usuario, e **ignora cualquier extensión** que pida el cliente: es la CA quien decide EKU y KeyUsage (el solicitante no puede concederse permisos). **Revocación:** el serial pasa a la CRL y se marca en la BD; el gateway detecta que la CRL cambió y recarga el contexto *sin reiniciarse* (`VERIFY_CRL_CHECK_LEAF`).

3.2 Canal TLS 1.3 y suites (RS2)

El contexto del servidor fija `minimum_version = maximum_version = TLSv1_3`: SSLv3, TLS 1.0, 1.1 y 1.2 quedan descatalogados (comprobado: un cliente que solo habla TLS 1.2 recibe la alerta *protocol_version*, ver §5). TLS 1.3 elimina los modos inseguros: no hay RSA estático, CBC, RC4 ni renegociación, y **todas** sus suites son AEAD; el intercambio de claves es siempre (EC)DHE, lo que da **Perfect Forward Secrecy**: aunque se robe después la clave privada del servidor, las sesiones grabadas no se pueden descifrar.

Suite	Cifrado autenticado	Hash (HKDF)	Política
TLS_AES_256_GCM_SHA384	AES-256 en modo GCM (AEAD)	SHA-384	**Permitida** (la que se negocia: preferencia del servidor)
TLS_CHACHA20_POLY1305_SHA256	ChaCha20-Poly1305 (AEAD)	SHA-256	**Permitida** (equipos sin aceleración AES)

Suite	Cifrado autenticado	Hash (HKDF)	Política
TLS_AES_128_GCM_SHA256	AES-128 en modo GCM (AEAD)	SHA-256	Fuera de política: negociable por OpenSSL, **cortada** tras el handshake (evento CIPHER_REJECTED)

Limitación declarada. El módulo `ssl` de Python no ofrece una llamada para recortar las suites de TLS 1.3 (`set_ciphers` solo gobierna TLS ≤ 1.2; con nombres TLS 1.3 falla: *No cipher can be selected*). Por eso el servidor impone su preferencia (`OP_CIPHER_SERVER_PREFERENCE`) y **rechaza tras el handshake** cualquier suite fuera de la lista. Lo demostramos con un cliente `openssl s_client` independiente: si solo ofrece AES-128 se negocia y el servidor lo corta; si ofrece ambas, se elige AES-256-GCM.

Cierre seguro (RF4). El cliente envía *LOGOUT*, el servidor confirma y ambos envían *close_notify*. Con `num_tickets = 0` no se emiten tickets de reanudación, así que al cerrar no queda ningún secreto con el que reabrir o descifrar la sesión: las claves efímeras se descartan.

3.3 mTLS y autenticación en dos capas (RF2)

Capa 1 (transporte). `verify_mode = CERT_REQUIRED`: el servidor *exige* certificado de cliente firmado por nuestra CA, vigente, con EKU clientAuth y no revocado; el cliente valida a su vez cadena y nombre del servidor (`check_hostname = True`). Es la autenticación mutua del enunciado. **Capa 2 (aplicación).** Usuario y contraseña, verificados con **Argon2id** (sal única por usuario, coste en memoria); además se exige **CN del certificado = usuario** y que ese certificado sea el vigente de ese usuario en la BD. Resultado: un carnet robado sin contraseña no entra (ataque 4) y un carnet no sirve para hacerse pasar por otro (ataque 5). Defensas adicionales: bloqueo temporal tras 5 fallos, mensaje de error *genérico* y verificación «señuelo» para usuarios inexistentes (no se filtra por tiempo si el usuario existe), y un semáforo que limita a 4 las verificaciones Argon2id simultáneas (cada una usa ≈64 MB; sin límite, 300 logins agotarían la RAM).

3.4 Inmutabilidad y auditoría (RF1, RS4)

SQLite incluye *triggers* `BEFORE UPDATE/DELETE` que abortan cualquier cambio de usuario, rol, fecha de alta o clave pública, y el borrado de usuarios y certificados. Solo se permite marcar un certificado como revocado (una vez). Cambiar una identidad exige revocar y emitir otra. El log de auditoría guarda en cada línea el SHA-256 de la anterior: editar o borrar una línea rompe la cadena y `verify_chain()` lo detecta (probado).

4. Manual de despliegue y uso

Todos los comandos se ejecutan desde la raíz del proyecto. Probado en Windows 10 con Python 3.10. La guía completa para desarrolladores es `GUIA.md`.

4.1 Requisitos e instalación

```
python --version                       # >= 3.10
pip install -r requirements-dev.txt        # cryptography, argon2-cffi, psutil, matplotlib, pytest, reportlab
# Opcional: OpenSSL (pruebas con cliente independiente) y Wireshark + Npcap (capturas)
```

4.2 Generar la PKI y precargar 300 usuarios (RS1, RF5)

```
python -m src.pki.preload --reset      # ~25 s: CA, certificado de servidor y 300 usuarios con certificado
openssl verify -CAfile certs/ca/ca.crt certs/clients/pdi_user1/pdi_user1.crt   # -> OK
```

Crea `certs/` y `data/usecure.db`: 100 usuarios por rol (*pdi_user1…100*, *pas_user1…100*, *inv_user1…100*), con contraseña de **prueba** común (`PRELOAD_PASSWORD` en `src/config.py`). Las claves privadas no se versionan.

4.3 Lanzar el servidor y el cliente (RF2, RF4)

```
# terminal 1
python -m src.server.gateway                 # TLS 1.3 + mTLS en :8443
# terminal 2
python main.py                               # opción 4 = demo automática, 5 = sesión interactiva
```

En la sesión interactiva se elige el alias del certificado, el usuario y la contraseña; comandos: *recurso <nombre>*, *logout* o cualquier texto (eco cifrado). Cada perfil solo accede a sus recursos.

4.4 Registrar un usuario nuevo y revocar certificados (RF1)

```
python -m src.server.enroll                # terminal 1: servicio de alta (:8444)
python main.py                              # terminal 2: opción 6 (usuario, rol, email, código de alta, contraseña)
python -m src.pki.admin revoke pdi_user1    # anula el carnet; el gateway recarga la CRL solo
```

4.5 Pruebas, capturas, ataques y benchmark (RS3, RS4)

```
python -m pytest                                      # 37 pruebas, ~7 s (PKI/BD/puerto temporales)
python -m tools.capture_evidence                          # 4 pcaps + análisis con tshark  -> evidences/pcap/
python -m attacks.run_attacks                             # 11 ataques + informe           -> evidences/attacks/
python -m benchmarks.load_test --sessions 300 --runs 3    # ~3 min, TLS frente a plano     -> evidences/bench/
python main.py   # opción 8: verifica la cadena de hashes del log de auditoría
```

4.6 Entrega

```
python -m tools.generate_memoria_pdf     # esta memoria
python -m tools.build_zip                    # PAI2-ST2.zip (lista blanca, sin claves privadas)
```

Reproducibilidad: el zip no incluye `certs/` ni la BD; quien lo descomprima ejecuta §4.1-4.2 y obtiene una PKI nueva. **Valores de demostración** (contraseña de precarga, código de alta) están en `src/config.py`; en producción se sustituirían.

5. Análisis de capturas de red y registros de auditoría (RS4)

Las capturas se generan con `dumpcap` sobre la interfaz de loopback (Npcap) y se analizan con `tshark` (`python -m tools.capture_evidence`, informe en `evidences/pcap/analysis.txt`). Se usa un mensaje secreto reconocible (*INFORME-CONFIDENCIAL-NOTAS-2026*) y se busca en los bytes capturados.

Captura	Paquetes	¿Secreto en claro?	¿Contraseña en claro?	Resultado
tls13_mtls_session	33	**NO**	**NO**	TLS 1.3, suite 0x1302 (AES-256-GCM); todo el diálogo cifrado
plain_session (comparación)	19	**SÍ**	**SÍ**	Usuario, contraseña y mensaje legibles: cualquiera en la red los lee
tls12_rejected	11	-	-	Alerta *protocol_version* (70); 0 registros de datos
no_client_cert	12	-	-	Handshake abortado y conexión cerrada por el servidor

5.1 Qué se observa en la sesión TLS 1.3

Handshake. Un *ClientHello* que anuncia *supported_versions* = *0x0304* (TLS 1.3) y las suites 0x1302/0x1303/0x1301, y un *ServerHello* que elige **0x1302 = TLS_AES_256_GCM_SHA384**. El campo de versión de la capa de registro vale 0x0303 por compatibilidad con intermediarios; la versión real está en la extensión. **Cifrado.** A partir del ServerHello todo son registros *Application Data* (tipo 23) de tamaño variable: en TLS 1.3 viajan cifrados incluso el certificado del servidor, la petición de certificado, el certificado del cliente y los mensajes de aplicación; no hay nada ininteligible que «resumir» por parte del atacante. **Confidencialidad.** Ni el secreto ni la contraseña aparecen en el fichero de captura (6804 bytes, 33 paquetes); en la captura sin cifrar aparecen íntegros (ejemplo siguiente, extraído de las propias capturas).

```
SIN cifrar (plain_session.pcapng)                                        TLS 1.3 (tls13_mtls_session.pcapng)
{"op": "LOGIN", "username": "pdi_user1", "password": "U-Secure#2026"}    16 03 03 00 7a 02 00 00 76 03 03 25 ...  (ServerHello)
{"status": "OK", "role": "PDI", "msg": "Bienvenido al gateway U-Secure [PDI]  17 03 03 00 63 1c 0d 23 d2 76 ec 71 ...  (datos cifrados)
{"op": "MSG", "data": "INFORME-CONFIDENCIAL-NOTAS-2026"}                 17 03 03 00 5b 45 3e 64 04 bd bf 22 ...  (datos cifrados)
```

Descifrado de demostración. Con el fichero de claves de sesión (`tls13_session.keylog`, generado solo para esa captura) Wireshark sí reconstruye el diálogo; sin él, no. Es la prueba de que el contenido es recuperable únicamente por quien posee las claves efímeras (PFS).

Alertas en TLS 1.3. En la captura *no_client_cert* no se ve el motivo del rechazo (*certificate_required*) porque, tras el ServerHello, las alertas también van cifradas. La causa queda en el log de auditoría (evento *HANDSHAKE_FAILED*). En cambio la alerta de versión (TLS 1.2) sí es visible porque se emite antes de cifrar.

5.2 Registro de auditoría

`evidences/logs/audit.jsonl`: una línea JSON por evento, con IP, usuario, CN y serial del certificado, versión TLS, suite y motivo, encadenada por SHA-256. Tras ejecutar capturas y ataques contiene **52 eventos** y la cadena es **íntegra**.

Evento	Nº	Significado
CONNECT	13	Conexión mTLS (con CN, serial, TLS y suite)
DISCONNECT	13	Desconexión con motivo (logout, connection_lost, auth_failed…)
HANDSHAKE_FAILED	7	Rechazo en el handshake: sin cert., CA ajena, revocado, TLS 1.2…
LOGIN_OK	5	Autenticación completa (capa 1 + capa 2)
LOGIN_FAILED	6	Contraseña errónea o usuario inexistente
UNAUTHORIZED	1	CN ≠ usuario, certificado no vigente o recurso fuera del perfil
ACCOUNT_LOCKED	1	Bloqueo por fuerza bruta
LOGOUT	4	Cierre explícito de sesión (RF4)

Los eventos prueban lo pedido por RS4: **conexión** (CONNECT), **desconexión** (DISCONNECT/LOGOUT) e **intentos no autorizados** (HANDSHAKE_FAILED, LOGIN_FAILED, UNAUTHORIZED, ACCOUNT_LOCKED).

6. Benchmark de escalabilidad y sobrecoste (RS3)

Método. `benchmarks/load_test.py` arranca el servidor en un **proceso aparte** (para medir su CPU y RAM), lanza **300 clientes a la vez**, cada uno con **su propio certificado y usuario**, y los hace esperar en una **barrera** hasta que todos están conectados y autenticados (así se demuestra la simultaneidad: 300/300 sesiones abiertas a la vez). Después cada cliente envía 20 mensajes de 256 B (latencia/RTT) y 10 de 32 KB (throughput) y hace logout. Se repite **3 veces** y se informa la **mediana**. El servidor **plano** y el **TLS** comparten el mismo código de sesión y de concurrencia: la diferencia medida es el coste del cifrado.

Métrica	Plano	TLS 1.3 + mTLS	Overhead	Rango entre ejecuciones (plano → TLS)
Sesiones OK / fallidas	300 / 0	300 / 0	—	—
Handshake sin carga, p50 (ms)	0.7	15.5	+2133 %	0.5–0.9 → 12.6–15.5
Conexión con 300 a la vez, p50 (ms)	223	2923	+1211 %	172–223 → 2328–2923
RTT 256 B p50 / p95 / p99 (ms)	54.8 / 77.5 / 107.7	62.2 / 105.5 / 187.9	p50 +13 %	p50: 33.8–60.5 → 62.2–77.6
Throughput (MB/s)	50.6	41.0	-19 %	48.7–57.6 → 37.1–41.4
Mensajes por segundo	2431	1968	-19 %	—
CPU del servidor por MB (ms/MB)	20	23	+12 %	15–20 → 20–24
CPU del servidor en conectar 300 (s)	102.6	107.5	+5 %	dominada por Argon2id
RAM del servidor, pico (MB)	47	62	+31 %	—

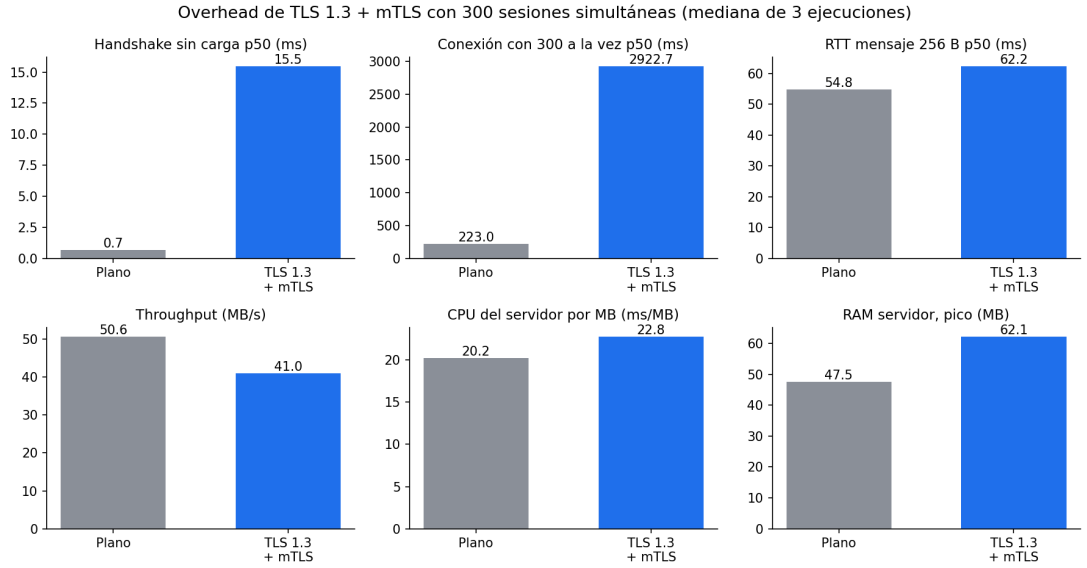


Figura 4. Plano frente a TLS 1.3 + mTLS con 300 sesiones simultáneas (mediana de 3 ejecuciones).

Interpretación

1. El coste está en el handshake, no en el cifrado. Una conexión aislada pasa de 0.7 a 15.5 ms: son las firmas ECDSA y el intercambio ECDHE, en *ambos* sentidos por mTLS. Con 300 conexiones simultáneas se forma cola y la mediana sube a 2923 ms (en el plano, 223 ms); es un pico puntual al iniciar la jornada, no un coste por mensaje. **2. Una vez abierto el túnel el sobrecoste es moderado:** la latencia de un mensaje sube un 13 % en la mediana (con mucha dispersión entre ejecuciones, ver rango), el throughput baja un 19 % y cada MB cuesta 12 % más CPU. **3. El login domina la CPU de conexión** (107.5 s frente a 102.6 s): Argon2id es caro a propósito y es idéntico en ambos modos, por eso el overhead de conectar es solo del +5 %. **4. Memoria:** +15 MB (+31 %) por el estado TLS de 300 sesiones.

Conclusión: el sistema soporta 300 sesiones simultáneas con 0 fallos. **Limitaciones honestas:** cliente y servidor comparten máquina (loopback), de modo que compiten por CPU y las cifras son *relativas*, no absolutas; el servidor Python está limitado por el GIL, por lo que el throughput absoluto no representa un despliegue real; y la dispersión entre ejecuciones es notable (por eso se informa la mediana y el rango). Entorno: 8 CPU, 15.7 GB RAM, Python 3.10.11.

7. Verificación con ataques simulados (extra opcional)

`python -m attacks.run_attacks` lanza 11 ataques en entorno controlado contra el gateway real (misma PKI y BD). Resultado: **11/11 bloqueados**. Cada uno se comprueba además en el log de auditoría para confirmar que lo rechazó el *servidor* y por el motivo esperado.

#	Ataque	Defensa	Resultado
1	Cliente sin certificado	mTLS obligatorio (CERT_REQUIRED)	**Bloqueado**
2	Suplantación con certificado de CA falsa	El servidor valida la cadena contra NUESTRA CA, no por el nombre	**Bloqueado**
3	Ataque de degradación a TLS 1.2	min = max = TLS 1.3 en el servidor	**Bloqueado**
4	Certificado robado, sin contraseña	Doble capa: además del carnet se exige la contraseña (Argon2id)	**Bloqueado**
5	Usar un carnet para entrar como otro usuario	El usuario debe coincidir con el CN del certificado	**Bloqueado**
6	Fuerza bruta de contraseñas	Bloqueo temporal de la cuenta	**Bloqueado**
7	Servidor falso (suplantación del gateway)	El cliente solo confía en la CA de U-Secure y valida el nombre (autenticación del servidor)	**Bloqueado**
8	Interceptación (sniffing) en un retransmisor MITM	Cifrado AEAD con TLS 1.3	**Bloqueado**
9	Manipulación de un bit en tránsito	Integridad AEAD (AES-GCM / ChaCha20-Poly1305): el registro alterado no se acepta	**Bloqueado**
10	Repetición (replay) de una sesión grabada	Claves efímeras (ECDHE) y aleatorios distintos por sesión: lo grabado no vale en otra sesión	**Bloqueado**
11	Uso de un certificado revocado	Comprobación de la CRL en el handshake	**Bloqueado**

Dos matices que importan. (a) En el ataque 10 (*replay*) el servidor *sí* responde con un ServerHello, pero **nuevo** (aleatorios y claves ECDHE distintos); al intentar descifrar los bytes grabados falla con *bad record mac* y no se abre ninguna sesión. Nuestra primera versión de la prueba lo marcó como fallo porque asumía que el servidor no contestaría; se corrigió el *criterio* tras leer el log, no la defensa. (b) En el ataque 9 se modifica un solo bit de un registro cifrado: la integridad AEAD lo detecta (*bad_record_mac*) y la sesión se corta.

Objetivo 5 (interceptación y suplantación en redes no confiables). El retransmisor MITM (`attacks/relay.py`) reenvía el tráfico y registra todo lo que pasa: no encuentra ni el secreto ni la contraseña (ataque 8). El servidor falso con certificado de una CA con el *mismo nombre* pero otra clave es rechazado por el cliente (ataque 7): se valida la cadena criptográfica, no el nombre.

8. Grado de completitud (trazable frente a RF y RS)

Estado: **Completo** = implementado, probado y con evidencia; **Con matiz** = cumple con una limitación declarada. Todas las rutas son relativas a la raíz del proyecto.

Req.	Enunciado	Estado	Implementación	Prueba / evidencia
RF1	Registro con usuario y contraseña, vinculado a la emisión de certificado	Completo	src/server/enroll.py · src/client/register.py · ca.sign_csr()	tests/test_enroll.py (5) · la clave privada no viaja
RF1	Inmutabilidad: identidad y clave pública no modificables sin re-emisión/revocación	Completo	src/server/store.py (triggers) · ca.revoke()	tests/test_store_audit_protocol.py (inmutabilidad ×5)
RF2	Doble capa: mTLS (dispositivo) + usuario/contraseña	Completo	src/server/gateway.py · session._authenticate() · security/kdf.py	tests/test_auth_session.py · ataques 4 y 5 · audit.jsonl
RF4	Cierre explícito, destrucción de claves efímeras y desmontaje inmediato	Completo	session._serve() LOGOUT · net.close_tls() · num_tickets=0	test_logout_closes_session… · evento LOGOUT/DISCONNECT · pcap (close_notify)
RF5	Precarga de cuentas PDI/PTGAS/Investigadores con certificados	Completo	src/pki/preload.py (300 usuarios) · perfiles en session.ROLE_RESOURCES	300/300 sesiones en el benchmark · test_role_based_resource_access
RS1	CA corporativa: ciclo de vida X.509 (servidor y dispositivos)	Completo	src/pki/ca.py (CA, CSR, CRL, revocación) · admin.py	openssl verify · test_revoked_certificate_is_rejected
RS1	mTLS: cliente valida servidor y servidor exige/comprueba certificado de cliente	Completo	gateway.build_server_context() CERT_REQUIRED · byod_client.build_client_context()	tests/test_tls_mtls.py · ataques 1, 2, 7, 11
RS2	TLS 1.3; SSLv3–TLS 1.2 descatalogados	Completo	min=max=TLSv1_3 (gateway, enroll, cliente)	test_obsolete_tls_versions_are_rejected · ataque 3 · tls12_rejected.pcapng
RS2	Solo Cipher Suites AEAD con PFS (AES-256-GCM / ChaCha20-Poly1305)	Con matiz	ALLOWED_CIPHERS + OP_CIPHER_SERVER_PREFERENCE + CIPHER_REJECTED	test_unauthorised_suite_… (openssl s_client) · matiz en §3.2
RS3	Modelo de concurrencia para 300 sesiones simultáneas	Completo	src/server/net.py (hilo por conexión) · semáforo Argon2id	benchmark: 300/300 simultáneas, 0 fallos

Req.	Enunciado	Estado	Implementación	Prueba / evidencia
RS3	Análisis comparativo throughput, RTT y CPU/RAM frente a tráfico sin cifrar	Completo	benchmarks/load_test.py · plain_server.py · plot.py	evidences/bench/ (JSON, PNG, informe) · §6
RS4	Capturas .pcap: tráfico 100 % cifrado en la interfaz pública	Completo	tools/capture_evidence.py	evidences/pcap/*.pcapng · analysis.txt · §5
RS4	Registros de conexión, desconexión e intentos no autorizados	Completo	src/server/audit.py (JSONL encadenado)	evidences/logs/audit.jsonl · verify_chain()
Obj. 5	Demostrar resistencia a interceptación y suplantación (+10 %)	Completo	attacks/run_attacks.py · attacks/relay.py	evidences/attacks/attacks_report.md · 11/11 bloqueados

Limitaciones conocidas: (1) las suites de TLS 1.3 no se recortan en el handshake sino que se rechazan tras él (§3.2); (2) benchmark en una sola máquina (§6); (3) la contraseña de precarga y el código de alta son valores de demostración; (4) las claves de usuario se guardan sin passphrase en disco (como en un entorno de laboratorio); (5) las claves privadas que estuvieron en el historial git de la versión inicial deben considerarse comprometidas y no se reutilizan (la PKI se regenera).

9. Reflexión sobre el uso de IA

Usamos **Claude Code** como asistente de ingeniería. El reparto fue deliberado: el equipo eligió la tecnología (sockets TLS), aprobó el plan y revisó cada bloque; la IA auditó la base de partida frente a la rúbrica, propuso el plan y generó el código, las pruebas y la documentación. La IA **no fijó el alcance** y no empezó a implementar hasta que el equipo aprobó el plan. El registro completo está en `AGENTS.md` (incluido en la entrega).

Cómo validamos el código generado

Nada se dio por bueno porque «compilara». (1) 37 pruebas escritas para *intentar romper* la defensa, no para confirmar el camino feliz. (2) Herramientas **independientes** del código generado: `openssl verify/x509/s_client` y `tshark`. (3) Ejecución real: procesos separados, 300 sesiones, 11 ataques. (4) Lectura del log de auditoría para confirmar que rechazó el servidor y por el motivo esperado.

Errores de la IA que detectamos y corregimos

Qué falló	Cómo lo detectamos	Corrección
Tres pruebas de rechazo de certificado fallaban: la IA esperaba un error al conectar	pytest	En TLS 1.3 el cliente termina el handshake antes de que el servidor valide su certificado. La prueba acepta excepción *o* cierre y además **exige** el evento en el log del servidor (más estricta, no más laxa)
El replay figuraba como «no bloqueado»	run_attacks	Era un fallo del **criterio** de la prueba, no de la defensa (ver §7). Se leyó el log y se documentó en vez de forzar el verde
El primer benchmark mostraba un +2657 % de handshake, cifra engañosa	Revisión crítica	Se añadió medición sin carga y se explica la cola de 300 handshakes; además se sustituyó el «% de CPU» (incoherente, TLS salía menor) por ms de CPU por MB
Se pretendía «forzar» suites TLS 1.3 con *set_ciphers*	Probando en el intérprete	No es posible en este Python; solución con preferencia + lista blanca y **prueba con openssl**; la limitación se declara
Una frase afirmaba que Python ≥ 3.13 lo resolvería	Revisión del texto	No se podía respaldar: se eliminó
300 logins simultáneos ≈ 19 GB de RAM (Argon2id ≈ 64 MB cada uno)	Cálculo previo	Semáforo de 4 verificaciones
tshark mostraba basura en lugar del texto descifrado	Revisar el informe	El campo era el texto cifrado: se usó *follow,tls* con keylog

Reflexión. La IA es muy eficaz generando estructura, pruebas y documentación, pero **tiende a dar por correcto lo que «debería» funcionar**: casi todos los errores aparecieron al *ejecutar*, no al leer el código. Un resultado desfavorable puede ser un fallo de la prueba y no de la defensa (el replay), y al revés; por eso hay que leer el log y no solo el semáforo verde/rojo. También aprendimos que la IA puede afirmar con seguridad cosas que no ha comprobado (la frase sobre Python 3.13): todo lo que aparece en esta memoria se ha ejecutado o se declara como limitación. La responsabilidad de lo entregado es del equipo.